

## 3008ICT Lab 8 (NLP)

### Task 1. Questions and Discussions

1. Increasing the size of  $n$  in an  $n$ -gram language model helps resolve data sparsity issues. True or False? Explain why.
2. A fixed-window neural language model solves the storage problem associated with  $n$ -gram models. True or False? Explain why.
3. Which of the following is a major drawback of using greedy decoding in sequence-to-sequence models?
  - a) It takes exponential time to compute the best path.
  - b) It requires finding the maximum probability of all possible sequences, which takes  $n^T$  time.
  - c) It has no way to undo decisions, meaning an early suboptimal choice cannot be corrected.
  - d) It requires you to track  $k$  independent hypotheses simultaneously.
4. When using a Recurrent Neural Network (RNN) for sentence classification (e.g., sentiment analysis), which approach usually yields the best results for computing the sentence encoding?
  - a) Using only the very first hidden state.
  - b) Using only the final hidden state of the network.
  - c) Multiplying the input embeddings directly without the hidden states.
  - d) Taking the element-wise max or mean of all hidden states.
5. The Window Problem: While a fixed-window neural language model improves upon  $n$ -gram models, the window size remains a limitation. Explain why we cannot simply make the fixed window infinitely large to capture long-term context.

6. Decoding Strategies: Explain the difference between Exhaustive Search and Beam Search in the context of Neural Machine Translation. Why is Beam Search the standard?

## Task 2. Programming in Pytorch

**Task 2.1.** The Sequence-to-Sequence Base Architecture: In the lectures, a conceptual structure for an Encoder-Decoder base class was introduced using a generic neural network block system.

Your Task: Translate this abstract structure into a functional PyTorch nn.Module base class. Fill in the missing code block below.

```
In [4]: 1 import torch
        2 import torch.nn as nn
        3
        4 class EncoderDecoder(nn.Module):
        5     """The base class for the encoder-decoder architecture."""
        6     def __init__(self, encoder, decoder, **kwargs):
        7         super(EncoderDecoder, self).__init__(**kwargs)
        8         # TODO: Initialize the encoder and decoder components
        9         self.encoder = None
       10         self.decoder = None
       11
       12     def forward(self, enc_X, dec_X, *args):
       13         # TODO:
       14         # 1. Pass the input sequence (enc_X) through the encode
       15         # 2. Initialize the decoder's state using the encoder's
       16         # 3. Pass the target sequence (dec_X) and the state thr
       17         # 4. Return the decoder's output.
       18         pass
```

**Task 2.2.** Pooling Hidden States for Classification: When doing sentence classification (like sentiment analysis), the basic method is to use the final hidden state to represent the sentence. However, a usually better method is to take the element-wise mean or max of all hidden states across the sequence. Your Task: Write a PyTorch function that takes a tensor of RNN hidden states and applies element-wise mean pooling to compute the final sentence encoding.

Assume `hidden_states` has the shape (batch\_size, sequence\_length, hidden\_dimension).

```
In [6]: 1 import torch
        2
        3 def compute_sentence_encoding(hidden_states):
        4     """
        5     Computes the sentence encoding by taking the mean of all hi
        6     across the sequence length.
        7
        8     Args:
        9         hidden_states: torch.Tensor of shape (batch_size, seq_l
10
        11     Returns:
        12         sentence_encoding: torch.Tensor of shape (batch_size, h
        13     """
        14     # TODO: Implement element-wise mean pooling over the sequen
        15     pass
```

```
In [ ]: 1
```